

# **MLflow**

---

## **Managing the Machine Learning Lifecycle**

MLOps Bootcamp

Instructor: Asadullah

# What is MLflow?

MLflow is an open-source platform designed to manage the end-to-end machine learning lifecycle. It tackles the complexities of developing, training, and deploying ML models.

## The Core Problem

Tracking experiments in local scripts is messy. Comparing different models, parameters, and metrics across a team is difficult without a standardized tool.

## The Solution

MLflow provides a unified API to **log parameters, metrics, and models**. It creates a structured repository for all your experiments.

# MLflow Components

MLflow consists of four primary modules:

- **MLflow Tracking:** Records and compares experiments (parameters, metrics, artifacts).
- **MLflow Projects:** Packages code in a reusable, reproducible format (like Docker/Conda).
- **MLflow Models:** Manages model deployment to various serving environments.
- **Model Registry:** A centralized store to manage the lifecycle of models (Versioning, Staging, Production).

# Setting Up MLflow

Install the MLflow library and launch the UI server locally:

```
pip install mlflow  
  
mlflow ui
```

Once executed, the MLflow server runs locally. Open your browser and navigate to:

```
http://127.0.0.1:5000
```

*\*Note: For team collaboration, this server should be hosted remotely (e.g., on AWS, Azure, or DAGsHub).*

# Core Tracking Concepts

## 1. Experiment

A logical grouping of runs. For example, a specific project like "Network-Security-Detection".

## 2. Run

A single execution of a model training script. Each run logs specific data.

## What goes inside a Run?

- **Parameters:** Key-value inputs (e.g., learning\_rate: 0.01).
- **Metrics:** Evaluation numbers that can update over time (e.g., accuracy: 0.95).
- **Artifacts:** Output files like the trained model (.pkl), confusion matrix plots, or datasets.

# Logging with Python

Here is how you integrate MLflow into a standard scikit-learn training script:

```
import mlflow
import mlflow.sklearn
from sklearn.ensemble import RandomForestClassifier

mlflow.set_experiment("Diabetes Prediction")

with mlflow.start_run():
    n_estimators = 100
    model = RandomForestClassifier(n_estimators=n_estimators)
    model.fit(X_train, y_train)

    accuracy = model.score(X_test, y_test)

    mlflow.log_param("n_estimators", n_estimators)
    mlflow.log_metric("accuracy", accuracy)
    mlflow.sklearn.log_model(model, "random-forest-model")
```

# The MLflow UI

When you run `mlflow ui`, you get a powerful dashboard:

- **Table View:** Lists all runs with their parameters and metrics.
- **Filtering & Sorting:** Easily find the best model by sorting by accuracy or filtering by specific parameters.
- **Chart View:** Compare multiple runs visually using scatter plots and parallel coordinates.
- **Artifacts Tab:** View the saved model structure and metadata for a specific run.

# Model Registry

Once you find the best model, you register it for deployment.

The Model Registry provides a centralized version control system for your models.

## Model Stages

- **None:** Initial registration.
- **Staging:** Model is in pre-production/testing.
- **Production:** Model is live and serving users.
- **Archived:** Model is retired.

## Benefits

Full lineage and audit trail. You know exactly which code, parameters, and data version created the model currently in production.

# Summary

## Key Takeaways

- **MLflow** standardizes the ML lifecycle.
- **Tracking** logs parameters, metrics, and artifacts for every run.
- The **UI** allows easy visual comparison of experiments.
- The **Model Registry** manages model versions and stages (Staging, Production).
- Can be run locally or hosted remotely for team collaboration.

Thank You for Watching!

Like & Subscribe to Asadullah AI